Project Info Extractor

Name: Lam Bing Qing Andrew
Student: s5
Unit: CFC060524
Trainer: Tushar

Table of Contents:

1. Introduction

2. Methodologies
    a. Commands
        - while loop
        - if/else conditions
        - exit
        - >
        - read
        - echo
        - pr
        - rm
        - sudo
        - updatedb
        - grep
        - awk
        - sort
        - head
        - curl
        - hostname
        - ifconfig
        - ps
        - top
        - service
        - locate
        - du

    b. Script objectives
        - Get user confirmation
        - Get user public IP address
        - Get user private IP address
        - Get device MAC address
        - Get top 5 processes using the most CPU on device
        - Get memory and swapfile statistics on device
        - Get all active services on device
        - Prompt user to update directory database
        - Get top 10 largest files in /home on device

3. Discussion

4. Conclusion

5. Recommendations

6. References

1. Introduction

This report aims to explain the commands used and how they were used to achieve the objectives in the accompanying script. It also aims to showcase the versatility of scripts and how useful and powerful they can be.

The script is configured to extract various information on the user's device and network after receiving confirmation to run.

2. **Methodologies**
    a. **Commands**

- `while` loop[1]

`while` loops are used when a command is required to run until a condition is fulfilled. The syntax in bash is:

```
while [[ condition ]]; do
     (action)

done
```

- if/else conditions[2]

If/else conditions are used when a command is only required to run if condition(s) has been fulfilled. The syntax is:

```
if [[ condition ]]; then
     (action)

elif [[ second condition ]]; then
     (action)

else
     (action)

fi
```

- `exit`[3]

`exit` is used when the script is required to terminate. It can also be used to return the result of a variable. The syntax is:

```
exit (variable)
```

- `>`[4]

`>` is used to save a command output into a file. It would overwrite anything in the file, so use with care. Use >> instead to append outputs to the end of the file. The syntax is:

```
(action) > file.*
```

- `read`[5]

`read` is used to get input from the user. The input is taken and saved into a variable. The syntax is:

```
read [option] [variable]
```

- `echo`[6]

`echo` is used to output text to the command line. The syntax is:

```
echo 'output'
```

- `pr`[7]

pr is used to prepare a file for printing by adding various formatting. It can also be used on a file such that the output achieves the desired format. The syntax is:

```
pr [option] file.*
```

- `rm`[8]

`rm` is used to delete files and directories. The syntax is:

```
rm [option] file.*
```

- `sudo`[9]

`sudo` is used when elevated permissions are required to run certain commands. Running `sudo` requires the user to be a super user. By default, `sudo` also requires users to authenticate themselves before running the command. The syntax is:

```
sudo [option] (command)
```

- `updatedb`[10]

`updatedb` is used to update the directory database. It requires `sudo` to run. The syntax is:

```
updatedb [option]
```

- `grep`[11]

`grep` is used to filter outputs and files with keywords or patterns. The syntax is:

```
grep [option] 'pattern' file.*
```

- `awk`[12]

`awk` is used to scan outputs and files, split its contents into rows, and filter and/or perform actions on them. The syntax is:

```
awk [option] 'pattern {action}' file.*
```

- `sort`[13]

`sort` is used to sort outputs by ascending or descending order. The syntax is:

```
sort [option] file.*
```

- `head`[14]

  `head` is used to display the top lines of an output or file. The syntax is:

```
head [option] file.*
```

- `curl`[15]

  `curl` is used to fetch data from URLs. The syntax is:

```
curl [option] 'URL'
```

- `hostname`[16]

`hostname` is used to obtain or set the system's hostname, DNS (Domain name system) name, or NIS (Network Information System) domain. The syntax is:

```
hostname [option] file.*
```

- `ifconfig`[17]

`ifconfig` is used to obtain information or configure the system network interfaces. The syntax is:

```
ifconfig [interface] [option]
```

- `ps`[18]

  `ps` is used to list running processes on the system. The syntax is:

```
ps [option]
```

- `top`[19]

  `top` is used to monitor system activity. The syntax is:

```
top [option]
```

- `service`[20]

`service` is used to start, stop, restart, or list the status of installed services on the device. The syntax is:

```
service <service name> [option]
```

- `locate`[21]

`locate` is used to find files with the system directory database using keywords or patterns. The syntax is:

```
locate [option] 'pattern'
```

- `du`[22]

`du` is used to analyze disk usage of directories and files on the system. The syntax is:

```
du [option] directory/file.*
```

b. **Script Objectives**

- Get user confirmation

```
3   #Loop input prompt till valid input is detected
4   while [[ $confirm != 'y' ]]; do
5       #Reading input for confirmation to run script
6       read -p 'Do you allow this script to run? (y/n):' confirm
7
8       #Exit script if user does not allow script to run
9       if [[ $confirm == 'n' ]]; then
10          echo 'Exiting'
11          exit
12
13      #Confirmation detected
14      elif [[ $confirm == 'y' ]]; then
15          echo 'Running script...'
16
17      #Prompting user that input was invalid
18      else
19          echo 'Invalid input!'
20
21      fi
22
23  done
```

First, I used `read` to prompt for user confirmation to run the script. The prompt is looped with `while` until a valid input is detected. Using if/else conditions, the script is configured to exit if it detects a `'n'` input, and run if a `'y'` is detected. All other inputs are rejected and the loop continues until a valid input is detected.

- Get user public IP address

```
25  echo
26  echo 'Your public IP address is:'
27  #Calling ifconfig.io to return public IP address
28  curl -s ifconfig.io
```

Using `echo`, I inform the user that the output is their public IP address. I grab the public IP address using `curl` to call the website ifconfig.io to return the public IP address.

- Get user private IP address

```
30  echo
31  echo 'Your internal IP address is:'
32  #Return internal IP address
33  hostname -I
```

I grab the private IP address of the device using `hostname -I`.

- Get device MAC address

```
35  echo
36  echo 'Your device MAC address is:'
37  #Calling ifconfig to return device network details, and filtering and censoring results to return the censored MAC address
38  ifconfig | grep ether | awk '{print $2}' | awk -F: '{print "XX:XX:XX:"$4":"$5":"$6""}'
```

The device MAC address is acquired using `ifconfig`. The result is passed through `grep` to filter for only the line containing the MAC address, then using `awk`, is further filtered for only the last 3 blocks of the MAC address. It is then output in its censored form.

- Get top 5 processes using the most CPU on device

```
40  echo
41  echo 'The top 5 processes using highest amount of CPU are:'
42  #Calling ps to return CPU usage and process name, sorting results by CPU usage, filtering for the top 5 results
43  #and inputting them into a temp file
44  ps -eo %cpu,comm | sort -nr | head -5 > Temp
45  #printing results from temp file and numbering them
46  pr -tn Temp
```

`ps` is called to list only the running processes' names and CPU usage. The results are then sorted by CPU usage and then using `head`, filtered so the output only shows the top 5 processes using the most CPU. The output is printed into a file called `Temp`, which I am using as a temporary file to store variables. Using `pr`, I then number the output by line.

- Get memory and swapfile statistics on device

```
48  echo
49  echo 'Here is the memory and swapfile statistics for your device:'
50  #Calling top and filtering for the memory and swapfile stats
51  top -b -n 1 | grep cache
52  top -b -n 1 | grep Swap
```

Using `top`, I list the system activity statistics. I then pass the output through `grep` to get the memory and swapfile statistics respectively.

- Get all active services on device

```
54  echo
55  echo 'Here is the list of all your active services:'
56  #Calling service to return the status of all services and filtering for the active services
57  service --status-all | grep +
```

The list of all installed services are called using `service --status-all`. The output is then passed through `grep` to get the list of active services only.

- Prompt user to update directory database

```
59  echo
60  echo 'Please enter user password to update directory database:'
61  #Updating directory database
62  sudo updatedb
```

The script prompts for the user password to update the directory database in preparation of the next step. `sudo updatedb` is used here.

- Get top 10 largest files in /home on device

```
64  echo
65  echo 'Here is the list of your top 10 largest files in the /home directory'
66  #Calling locate to return all files under /home, passing result through du to return file size, sorting and filtering for top 10 largest files,
67  #and inputting results into temp file
68  locate /home | du * | sort -nr | head -10 > Temp
69  #printing results from temp file and numbering them
70  pr -tn Temp
```

Using `locate`, I search for a complete list of all files in the `/home` directory. I then find the file sizes of all of them by using `du`. The result is then sorted and filtered to show the 10 largest files. The output is then saved into `Temp`. I then use `pr` to number the output.

3. Discussion

The script shown above requires minimal user input to grab plenty of various information on the user's device. As seen, scripting is a very powerful tool that can be used to automate for various purposes, including nefarious ones. As such, care and caution should be taken when running scripts of unknown origin. Always take a moment to audit the code in the script if possible.

On the other hand, people writing the scripts should also exercise due care and only have the script perform its intended purpose. Collecting too much information through scripts can lead to unintended breaches of the user's privacy or leakage of classified or sensitive information.

4. Conclusion

Scripts are a powerful tool. But it is just that, a tool. To use it for good or bad is up to the individual. As such, all the rest of us can do is to exercise due care when running or writing scripts.

5. Recommendations

- Audit the code of scripts before you run them.
- Do not collect excessive amounts of information for ‘just in cases’. Just have your written scripts do its intended function.

6. References

1. ↑ *While(1): Perform Command Multiple Times - Linux man page*. Available at: https://linux.die.net/man/1/while (Accessed: 03 June 2024).
2. ↑ *If(1): Conditionally execute command - linux man page*. Available at: https://linux.die.net/man/1/if (Accessed: 03 June 2024).
3. ↑ *Exit(1): Bash built-in commands, see bash - linux man page*. Available at: https://linux.die.net/man/1/exit (Accessed: 03 June 2024).
4. ↑ *Simple redirections*. Available at: https://linux.die.net/Intro-Linux/sect_05_01.html (Accessed: 03 June 2024).
5. ↑ *Read(3): Read from file - linux man page*. Available at: https://linux.die.net/man/3/read (Accessed: 03 June 2024).
6. ↑ *Echo(1): Line of text - linux man page*. Available at: https://linux.die.net/man/1/echo (Accessed: 03 June 2024).
7. ↑ *PR(1): Convert text files for printing - linux man page*. Available at: https://linux.die.net/man/1/pr (Accessed: 03 June 2024).
8. ↑ *RM(1): Remove files/directories - linux man page*. Available at: https://linux.die.net/man/1/rm (Accessed: 03 June 2024).
9. ↑ *SUDO(8): Execute command as another user - linux man page*. Available at: https://linux.die.net/man/8/sudo (Accessed: 03 June 2024).
10. ↑ *UPDATEDB(8): Update database for mlocate - linux man page*. Available at: https://linux.die.net/man/8/updatedb (Accessed: 03 June 2024).
11. ↑ *GREP(1): Print Lines Matching Pattern - Linux man page*. Available at: https://linux.die.net/man/1/grep (Accessed: 03 June 2024).
12. ↑ *AWK(1): Pattern scanning/processing - linux man page*. Available at: https://linux.die.net/man/1/awk (Accessed: 03 June 2024).
13. ↑ *Sort(1): Sort lines of text files - linux man page*. Available at: https://linux.die.net/man/1/sort (Accessed: 03 June 2024).
14. ↑ *Head(1): Output first part of files - linux man page*. Available at: https://linux.die.net/man/1/head (Accessed: 03 June 2024).
15. ↑ *Curl(1): Transfer URL - linux man page*. Available at: https://linux.die.net/man/1/curl (Accessed: 03 June 2024).
16. ↑ *Hostname(1): Show/set system's host name - linux man page*. Available at: https://linux.die.net/man/1/hostname (Accessed: 03 June 2024).
17. ↑ *Ifconfig(8): Configure network interface - linux man page*. Available at: https://linux.die.net/man/8/ifconfig (Accessed: 03 June 2024).
18. ↑ *PS(1): Report snapshot of Current Processes - Linux man page*. Available at: https://linux.die.net/man/1/ps (Accessed: 03 June 2024).
19. ↑ *Top(1): Tasks - linux man page*. Available at: https://linux.die.net/man/1/top (Accessed: 03 June 2024).

20. ↑ *Service(8): Run system V init script - linux man page*. Available at: https://linux.die.net/man/8/service (Accessed: 03 June 2024).
21. ↑ *Locate(1): Find files by name - linux man page*. Available at: https://linux.die.net/man/1/locate (Accessed: 03 June 2024).
22. ↑ *DU(1): Estimate file space usage - linux man page*. Available at: https://linux.die.net/man/1/du (Accessed: 03 June 2024).